SAVEETHA SCHOOL OF ENGINEERING
C PROGRAMMING ASSIGNMENT – Grama Jyothi Billing Program

Field	Entry
Student Name	BAVADARANI S
Register Number	1912260562
Course Code & Name	CSA0209 C PROGRAMMING
Slot	B
Assignment / Task Title	Design, Implement and Validate a Reliable Billing Program
GitHub Repository URL	https://github.com/bavadarini28 /CSA0209-1912260562-BAVADARANI.S.git
Submission Date	12 NOVEMBER 2026

TASK 12 – Re-evaluate and Adapt the Design (Smart Meters)

A. Impact Analysis

- Readings arrive every 15 minutes in watt-hours: 35,040 per house per year (365 × 96) instead of one per month.
- A time-of-day rule applies a 20% surcharge on units used 6 PM–10 PM (slots 72–87 of the 96 daily).
- Tariff values must be changeable without recompiling.
- Types no longer sufficient: unsigned short for readings (65,535 Wh is only 65.5 kWh), 32-bit int for yearly/district Wh totals. Constants no longer fixed by #define: slab limits, rates, fixed charge and the peak percentage.

B. Evaluation of the Original Design

Largest values: assuming a 10 kW maximum connection, one reading ≤ 10,000 W × 0.25 h = 2,500 Wh. Per house per year: 35,040 × 2,500 = 87,600,000 Wh (fits in a 32-bit int, 2.1×10^9). District (800,000 houses): 70,080,000,000,000 Wh = 7.0×10^13, which exceeds even UINT32_MAX (4.29×10^9) – so 64-bit is mandatory. Energy charge expressed in paise × Wh: at Rs 5.40/kWh peak rate (540 paise per kWh) the district yearly bound is 3.78×10^16, below INT64_MAX (9.22×10^18), so int64_t is safe.

Original choice	Still appropriate?	Reason
unsigned short meter_t	No	Wh readings and yearly totals overflow 16 bits (65.5 kWh).
paise in long	Partly	Paise are too coarse: 1 Wh at Rs 4.50/kWh = 0.45 paise. Need paise × Wh (or milli-paise) with one final rounding.
#define tariff	No	Must be changed without recompiling.
Parenthesised macros	Yes	Still valid; but prefer inline functions.
Slab if-statements	Retain	Logic still valid when applied to monthly kWh.

C. Adaptation: MODIFY (not redesign from scratch)

Retained: integer arithmetic, one rounding step, slab logic, test table. Modified: int64_t for Wh and charges; tariff read from tariff.cfg into a struct; a peak-hour rule; rates in paise per kWh and working in paise × Wh,

rounded once to paise at the end. Assumption for the peak rule: the 20% surcharge applies to the average energy rate multiplied by the peak share of consumption.

```
#include <stdio.h>
#include <stdint.h>
typedef struct { int64_t s1, s2, r1, r2, r3, fixed, peak_pct; } tariff_t; /* rates: paise per kWh */
static int load_tariff(const char *f, tariff_t *t)
{
   FILE *fp = fopen(f, "r");
   if (!fp) return 0;
   int ok = fscanf(fp, "%ld %ld %ld %ld %ld %ld %ld", &t->s1,&t->s2,&t->r1,&t->r2,&t->r3,&t->fixed,&t->peak_pct) == 7;
   fclose(fp);
   return ok;
}
/* Returns bill in paise. wh_total, wh_peak in watt-hours (64-bit). Work in "paise x Wh" and round once. */
static int64_t bill(const tariff_t *t, int64_t wh_total, int64_t wh_peak)
{
   int64_t s1 = t->s1 * 1000, s2 = t->s2 * 1000;          /* slab limits in Wh */
   int64_t e = 0, w = wh_total;
   if (w > s2) { e += (w - s2) * t->r3; w = s2; }
   if (w > s1) { e += (w - s1) * t->r2; w = s1; }
   e += w * t->r1;                                        /* paise x Wh */
   int64_t surcharge = wh_total ? e * wh_peak * t->peak_pct / (wh_total * 100) : 0; /* average-rate method */
   int64_t energy = (e + surcharge + 500) / 1000;          /* single rounding to paise */
   return t->fixed + energy;
}
int main(void)
{
   tariff_t t;
   if (!load_tariff("tariff.cfg", &t)) { puts("cannot read tariff.cfg"); return 1; }
   int64_t p = bill(&t, 350000, 0);
   printf("350 kWh, no peak   : Rs %ld.%02ld\n", p/100, p%100);
   p = bill(&t, 350000, 100000);
   printf("350 kWh, 100 peak  : Rs %ld.%02ld\n", p/100, p%100);
   p = bill(&t, 150000, 150000);
   printf("150 kWh, all peak  : Rs %ld.%02ld\n", p/100, p%100);
   int64_t max_wh_house = (int64_t)35040 * 2500;   /* 10 kW x 0.25 h = 2500 Wh per reading */
   printf("max Wh/house/yr = %ld ; district = %ld ; INT32_MAX=%d ; UINT32_MAX=%u\n", max_wh_house,
max_wh_house*800000, INT32_MAX, UINT32_MAX);
   printf("district yearly charge bound (paise x Wh) = %ld ; INT64_MAX=%ld\n", max_wh_house*800000*540, INT64_MAX);
   return 0;
}
```

tariff.cfg: 100 200 0 225 450 5000 20

Output:

```
350 kWh, no peak   : Rs 950.00
350 kWh, 100 peak  : Rs 1001.43
150 kWh, all peak  : Rs 185.00
max Wh/house/yr = 87600000 ; district = 70080000000000 ; INT32_MAX=2147483647 ; UINT32_MAX=4294967295
district yearly charge bound (paise x Wh) = 37843200000000000 ; INT64_MAX=9223372036854775807
```

Checks by hand: 350 kWh without peak = Rs 950.00 (same as Task 3, so the old results are preserved). 100 of 350 kWh at peak: energy charge Rs 900 × (100/350) × 20% = Rs 51.43, total Rs 1001.43. 150 kWh all peak: energy 50 × 2.25 = Rs 112.50 × 1.2 = Rs 135.00, plus Rs 50 = Rs 185.00.

Programiz
C Online Compiler

Install

main.c

Clear Run ▶

```c
#include <stdio.h>
#include <stdint.h>
#include <inttypes.h>
#include <limits.h>

typedef struct
{
    int64_t s1, s2, r1, r2, r3;
    int64_t fixed, peak_pct;
} tariff_t;

/* Calculate electricity bill in paise */
static int64_t bill(const tariff_t *t,
                    int64_t wh_total,
                    int64_t wh_peak)
{
    int64_t s1 = t->s1 * 1000;
    int64_t s2 = t->s2 * 1000;

    int64_t e = 0;
    int64_t w = wh_total;
```

Output

```
Enter slab1 slab2 rate1 rate2 rate3 fixed peak_pct:
100 200 0 225 450 5000 10
350 kWh, no peak : Rs 950.00
350 kWh, 100 peak : Rs 975.71
150 kWh, all peak : Rs 173.75
Max Wh/house/year = 87600000
District = 70080000000000
INT32_MAX = 2147483647
UINT32_MAX = 4294967295
District yearly charge bound (paise x Wh) = 37843200000000000
INT64_MAX = 9223372036854775807
```

D. Evidence-Based Defence

- Calculation: 7.0×10^13 Wh per district-year proves 32-bit types are inadequate while int64_t has about 240× headroom (3.78×10^16 vs 9.22×10^18).
- Tests: with peak = 0 the new program reproduces the original Rs 950.00 bill, so the redesign is backward-compatible; peak cases agree with hand calculation.
- Trade-offs: a file-based tariff removes recompilation but needs validation and file distribution; integer paise × Wh keeps exactness at the cost of less readable code; storing all 35,040 readings per house per year costs about 35,040 × 8 bytes ≈ 280 KB per house, so aggregate into peak and total Wh per month as readings arrive rather than storing them all.
- Decision: MODIFY – keep the integer-only, one-rounding design, widen the types, move the tariff to a configuration file and add the peak rule.

Feedback about CO1

- Through this CO1 assignment, I gained a better understanding of fundamental C programming concepts, including data types, variables, constants, macros, header files, input and output operations, type conversion, and the compilation process.
- By completing all 12 tasks, I improved my programming, logical thinking, problem-solving, and debugging skills. I also learned how to identify errors, correct them, and develop programs that produce the expected output.
- Some tasks were challenging, but practicing them helped me improve my confidence in programming. Overall, this course outcome helped me strengthen my basic programming knowledge and apply it to practical problems.